Data structure

→ structure means particular way of data organization, so data structure is refers to the organization of data in computer memory or the way in which data is effiuently stored, processed & retrieved is called data structure.

→ Data structure is the structural representation of logical relationship b/w elements of data.

There are different kinds of data structures. some are used to store the data of same type and some are used to store different types of data.

→ Data structure not only deals with raw data but also involves the relationship b/w data.

Types of data structure →

(1) According to nature of size →

(a) static data structure :→ A data structure is said to be static if we can store data upto a fix number eg. array.

(b) Dynamic data structure :→ A dynamic data structure is that which allow the programmer to change its size during the execution of program to add or delete data.

THINGS TO DO

Scanned by CamScanner

(2) According to its occurrence

(a) Linear DS :- In Linear D.S. data is stored in consecutive memory location or data is stored in sequential form eg. array, link-list, stack, queue etc.

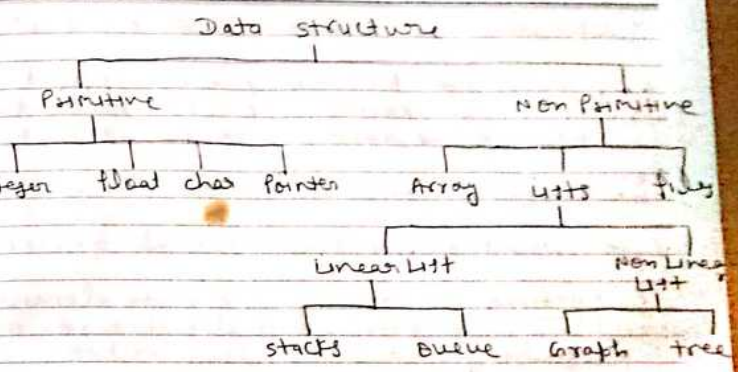
(b) Non Linear D.S. → In non Linear D.S. the data is stored in non consecutive memory location or data is stored in inconsequential form eg. tree, graph etc..

(3) Primitive and Non Primitive D.S.

→ Primitive D.S. are the basic D.S. and are directly operated upon by the machine instructions. These have different representations on different computers. Integers, floating point nos., char., string constants, pointers etc. are comes in this category.

→ Non Primitives D.S. are the more complicated D.S. emphasing on structuring of a group of homogeneous & heterogeneous data items. Array, list, files, link-list, tree, graph, fall in this category.

3 SUNDAY
THINGS TO DO



(4) Homogeneous & Non Homogeneous D.S.

In homogeneous D.S. the data elements are of same type like array, in non Homogeneous D.S. the data elements may not be of same type such as structure & Union etc..

Data structure operations

1. creating This is the first operation to create the D.S. This is just declaration and initialization of the D.S.
2. Inserting Adding new record to the S.
3. Deleting Removing a Record from S.

S → structure.

THINGS TO DO

Scanned by CamScanner

4. Updating It changes data values of D.S

5. Traversing Accessing each record exactly once so that certain items in the record may be processed

6. Searching Finding the location of the record with a given key value, or finding the location of all records.

7. Sorting Arranging the data elements in some logical order ie. in ascending and descending order.

8. Merging Combine the data elements in two different sorted sets into a single sorted set.

9. Destroying This is the last operation of D.S. and apply this operation when no longer needs of data.

Algorithm An algorithm is a step by step finite sequence of instructions, to solve a computational problem. Any algorithm have the following features :-

THINGS TO DO

→ It should be free from ambiguity.
→ It should be concise.
→ It should be efficient.

each & every algorithm must satisfy the following criteria :-

(1) Input There are zero or more inputs which are externally supplied.

(2) Output At least one o/p is produced.

(3) Definiteness Each instruction must be clear.

(4) Finiteness Algorithm should be terminate after a finite no. of steps.

(5) Effectiveness It means that operations must be simple & carried out in a finite time at one or more levels of complexity.

Array An array can be defined as the collection of the sequential memory location or consecutive memory locations which can be reffered by a single name alongwith a number known as the index to access a perticular field or data.
Each element of an array is referenced by a subscripted variable or value called subscript or index enclosed in parenthesis. If an element of an array is referenced by single subscript then the array is known as single or one dimensional array or linear array and if 2 subscripts

THINGS TO DO

Scanned by CamScanner

07 2010 THURSDAY • JANUARY

are required to be referenced an element, the array is known as 2D array and so on. The array whose elements are referenced by 2 or more subscripts are called multi-Dimensional array.

Declaration

type varname [size];

Q Print all elements of 1D array.

Q Wap to count +ve, -ve nos in array

Representing 1D array in memory

(1) For calculate the no. of elements in 1D array or for calculate the length of 1D array →

Length = UB - LB + 1

(2) For calculate the address of any element →

THINGS TO DO

LOC(A[K]) = base(A) + w(K - LB)

LOC(A[K]) = base(A) + w * K.

08 2010 JANUARY • FRIDAY

Q1 Base address of first element = 400
each element occupies = 2 bytes
Find the address of 4th element of array A[10].

Sol Address of 4th element =

base(A) + w * K

w = 2 bytes & K = 4

= 400 + 2*4 = 408

Q2 Base address of first element = 300
w = 4 bytes
→ Find the no. of elements in linear array A(5:50), B(-5:10), C(18)
→ Find the address of A[15], A[40] and A[55].

Sol (I) Length = UB - LB + 1

so Length(A) = 50 - 5 + 1 = 46

Length(B) = 10 - (-5) + 1 = 16

Length(C) = 18 - 1 + 1 = 18 (LB = 1)

THINGS TO DO (II) LOC(A[K]) = base(A) + w(K - LB)

LOC(A[15]) = 300 + 4(15 - 5) = 340

LOC(A[40]) = 300 + 4(40 - 5) = 440

A[55] is not an element of A because its exceeds UB = 50.

Scanned by CamScanner

09 - 2010
SATURDAY • JANUARY

operations on Array

(1) Traversing

Algorithm

1. I ← LB
2. For I = LB to UB
   Print A[I]
3. Stop.

(2) Insertion in 1D array

(a) At the beginning of the array :-

1. If UB = Max then write "Array is overflow and STOP"
2. Read Data
3. K ← UB
4. Repeat step(5) while K ≥ LB
5. A(K+1) ← A(K)
   K ← K-1
6. A(LB) ← Data
7. STOP.

10 SUNDAY

THINGS TO DO

11 - 2010
JANUARY • MONDAY

(b) At the end of Array :-

1. If UB = Max then write array is overflow and STOP
2. Read data
3. UB ← UB+1
   A[UB] ← Data
4. STOP.

(c) At the given location :-

1. If UB = Max then write "array is overflow and STOP"
2. Read data
3. Read LOC (LOC is a location where insertion will be made)
4. K ← UB
5. Repeat step(6) while K ≥ LOC
6. A(K+1) ← A(K)
   K ← K-1
7. A(LOC) ← Data
8. STOP.

THINGS TO DO

Scanned by CamScanner

12 2010 TUESDAY • JANUARY

(d) In the sorted Array :-

1. If UB = max then write " array is overflow and STOP.
2. Read data
3. K ← LB
4. Repeat step(5) while A(K) < Data
5. K ← K + 1
6. LOC ← K
   K ← UB
7. Repeat step(8) while K ≥ LOC
8. A(K+1) ← A(K)
   K ← K-1
9. A(LOC) ← data
10. STOP.

THINGS TO DO

2010 JANUARY • WEDNESDAY 13

(3) Deletion in 1D array

(a) Deletion from the end :-

1. If N = 0 then array is underflow and STOP.
2. A(UB) ← NULL
   UB ← UB - 1
3. STOP.

(b) Deletion from beginning :-

1. If N = 0 then array is underflow and STOP.
2. K ← LB
3. Repeat step(4) while K < UB
4. A(K) ← A(K+1)
5. A(UB) ← NULL
   UB ← UB - 1
6. STOP.

THINGS TO DO

Scanned by CamScanner

14 2010 THURSDAY • JANUARY

(C) Deletion of given element :-

1. If N=0 then array is underflow & stop.
2. Read data. (as element to be deleted)
3. Read LOC (as location where deletion will be made)
4. K ← LB
5. Repeat step (6) while A(K) ≠ data
6. K ← K+1
7. Repeat step (8) while K < UB
8. A(K) ← A(K+1)
   K ← K+1
9. A(UB) ← NULL
   UB ← UB-1
10. STOP.

THINGS TO DO

15 2010 JANUARY • FRIDAY

(4) Concatenation

1. K ← LB A
2. Repeat step (3) while K ≤ UB A
3. C(K) ← A(K)
   K ← K+1
4. L ← LB B
5. Repeat step (6) while L ≤ UB B
6. C(K) ← B(L)
   L ← L+1
   K ← K+1
7. stop.

(5) Inversion :-

1. I ← LB, J ← UB
2. Repeat step (3) and (4) while I < J

THINGS TO DO

3. Temp ← A(I)
   A(I) ← A(J)
   A(J) ← Temp

Scanned by CamScanner

16 2010
SATURDAY • JANUARY

4. I ← I+1
   J ← J-1
5. stop.

(6) merging

1. I ← LBA, J ← LBB, K ← 0
2. Repeat step (3) and (4) while I ≤ UBA and J ≤ UBB
3. If A(I) < B(J) then
   C(K) ← A(I)
   I ← I+1
   else
   C(K) ← B(J)
   J ← J+1
4. K ← K+1

17 SUNDAY
5. Repeat (6) while I ≤ UBA
THINGS TO DO
6. C(K) ← A(I)
   I ← I+1
   K ← K+1
7. Repeat step (8) while J ≤ UBB

18 2010
JANUARY • MONDAY

8. C(K) ← B(J)
   J ← J+1
   K ← K+1
9. stop.

Limitations of Linear Array

(1) Linear Array are static structures i.e. memory used by them can not be reduced or extended
(2) Previous knowledge of no. of elements in array is necessary.
(3) since the elements of these arrays are stored in consecutive locations, the insertion and deletions are time consuming.

Representation of 2D array in memory

The element of 2D array are stored in computer's memory row by row or column by column. If the array is stored as row by row, it is called row major order. If the array is stored as col by col., it is called column major order.
THINGS TO DO

Scanned by CamScanner

19 2010 TUESDAY • JANUARY

Under Row major representation, the first row occupies the first set of memory locations, the second row occupies the next set and so on. Under column major representation, the first column occupies the the first set of memory location, the second column occupies the next set and so on.

In Row major order

A11	A12	A13	first set
A21	A22	A23	2nd
A31	A32	A33	3rd

In column major order

A11, A21, A31, A12, A22, A32, A31, A32, A33 - - -

Q show how the matrix

2	5	8
1	7	6
3	4	9

would appear in memory when stored in

→ Row major

→ column major

Things To Do

Sol Row major order

2 5 8 1 7 6 3 4 9

col major order

2 1 3 5 7 4 8 6 9

20 2010 JANUARY • WEDNESDAY

Q A 2D array with 2 rows & 3 cols stand in memory as →

8 5 3 7 2 6

show the matrix form of 2D array.

Sol RMO

8 5 3

7 2 6

CMO

8	3	2
5	7	6

Address calculation for 2D Array

(a) column major order

(i) LOC(A[i][j]) = Base(A) + w[m(j - LB for col index) + (i - LB for row index)]

(ii) LOC(A[i][j]) = Base(A) + w[mj + i]

(b) Row major order

Things To Do (i) LOC(A[i][j]) = Base(A) + w[n(i - LB for row index) + (j - LB for col index)]

(ii) LOC(A[i][j]) = Base(A) + w[nxi + j]

Scanned by CamScanner

21 — 2010 — THURSDAY • JANUARY

Q(1) Each element of an array Data[20][50] requires 4 bytes. Base address is 2000. Find the LOC of Data[10][10] when the array is stored in →

→ Row major
→ col major

Sol Here data[20][50]

So m = 20, n = 50
w = 4 bytes

We want to find data[10][10]
So i = 10, j = 10

suppose first element of array is data[1][1].

(i) Row major →

LOC[Data[10][10]] = Base[data] + 4[50(10-1) + (10-1)]
= 2000 + 4[50 × 9 + 9]
= 3836.

(ii) Col major →

LOC[Data[10][10]] = Base[data] + 4[20(10-1) + (10-1)]
= 2000 + 4[20 × 9 + 9]
= 2756

THINGS TO DO

22 — 2010 — JANUARY • FRIDAY

Q(2) Array A[6][6]
first element is stored at LOC 100.
w = 4 bytes.
find the LOC A[2][5] if array is stored row wise.

Sol

LOC[A[2][5]] =
100 + 4[6(2-1) + (5-1)]
= 140.

Q3 Calculate the address of X[4,3] in a 2D array X[1..5, 1..4] stored in row major. Base address is 1000. & each element required 4 Bytes or words of storage.

Base (X) = 1000
w = 4
n = 4
X[4,3] = 1000 + 4[4(4-1) + (3-1)]
= 1056

THINGS TO DO

Scanned by CamScanner

23 | 2010 | SATURDAY • JANUARY

Traversing 2D array

column major order

1. for J = 0 to col - 1
2. for I = 0 to ROW - 1
3. Print A[I][J]
4. stop.

ROW major order

1. for I = 0 to ROW - 1
2. for J = 0 to col - 1
3. Print A[I][J]
4. stop.

Addition subtraction

1. for I = 0 to m - 1
2. for J = 0 to n - 1
   C[I][J] = A[I][J] + B[I][J]
3. stop.

24 SUNDAY

THINGS TO DO

Transpose

1. For I = 0 to M - 1
2. for J = I + 1 to n - 1

25 | 2010 | JANUARY • MONDAY

   temp = A[I, J]
   A[I, J] = A[J][I]
   A[J, I] = temp
3. stop

Multiplication

1. for i = 0 to m - 1
2. for j = 0 to n - 1
3. for k = 0 to p - 1
   c[i, j] = c[i, j] + A[i, k] * B[k, j]
4. stop.

special matrices or forms of sparse matrices

(1) Diagonal

$$\begin{bmatrix} 8 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 5 \end{bmatrix}$$

A[i, j] = 0 for i ≠ j

(2) Lower Triangular

$$\begin{bmatrix} 8 & 0 & 0 & 0 \\ 2 & 6 & 0 & 0 \\ 3 & 7 & 9 & 0 \\ 4 & 2 & 5 & 1 \end{bmatrix}$$

A[i, j] = 0 for i < j

Scanned by CamScanner

(3) Upper Triangular

$A[i,j] = 0$ for $i > j$

$$\begin{bmatrix} 5 & 3 & 8 & 9 \\ 0 & 2 & 4 & 7 \\ 0 & 0 & 5 & 1 \\ 0 & 0 & 0 & 6 \end{bmatrix}$$

(4) Tridiagonal

$A[i,j] = 0$ for $|i-j| > 1$

$$\begin{bmatrix} 3 & 2 & 0 & 0 \\ 5 & 4 & 6 & 0 \\ 0 & 1 & 9 & 7 \\ 0 & 0 & 4 & 8 \end{bmatrix}$$

Sparse Matrix An m×n matrix A is said to be sparse if many of its elements are zero. A matrix that is not sparse is called the dense matrix. It is not possible to define an exact boundary b/w sparse & dense matrices.

Ex

$$\begin{bmatrix} 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 5 \\ 0 & 0 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 9 & 0 & 0 & 0 \end{bmatrix}$$

THINGS TO DO

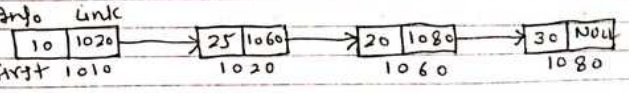
Linked List

A linked list is an ordered collection of finite homogeneous data elements called nodes where the linear order is maintained by pointers. That is each node is divided into 2 parts. The first part contains the information of element and the second part called the link field or the next pointer field, contain the address of the next node in the list.

The no. of pointers is maintained depending on the requirements and usage. Based on this linked lists are classified into following categories :-

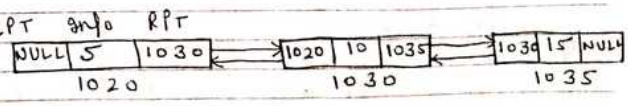
(1) Linear Linked list
(2) Doubly linked list
(3) Circular linked list
(4) Header linked list
(5) Circular doubly linked list

(1)



THINGS TO DO

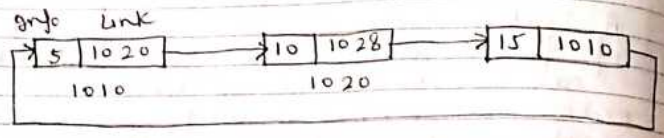
(2)



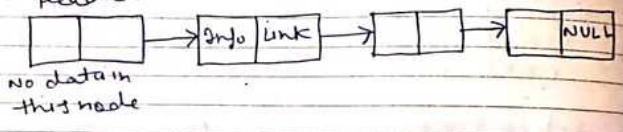
Scanned by CamScanner

28 2010
THURSDAY • JANUARY

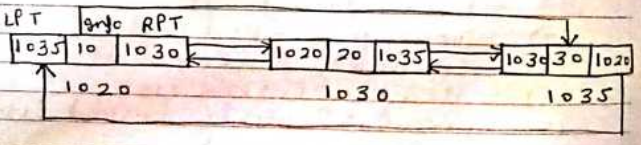
(3)



(4)



(5)



Advantages & Disadvantages of using Linked List

THINGS TO DO

Advantage (1) Linked list are Dynamic Data structure. They can extend or reduce at run time.

JANUARY

S	M	T	W	T	F	S
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					

FEBRUARY 2010

S	M	T	W	T	F	S
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28						

2010 Wk 05
JANUARY • FRIDAY 29

(2) Size is not fixed.

(3) Data can store non-continuous memory blocks.

(4) Insertion & deletion of nodes are easier and efficient.

(5) Many more complex applications can be easily carried out with linked list.

Disadvantages

(1) In the linked list, there is an special field called link field which holds the address of next node, so linked list requires extra space.

Creation of Singly Linked List

step1. Ptr = Avail
Avail = Link(Avail)
Read Info(Ptr)
First = Ptr
step2. ch = 'Y'
3. Repeat step 4 to 5 while ch = 'Y'
4. cPt = Avail
Avail = Link(Avail)
Read Info(cPt)
Link(Ptr) = cPt
Ptr = cPt
5. Read choice Y/N into ch for more nodes
6. Link(Ptr) = NULL
7. stop.

circular

6. Link(Ptr) = First

Scanned by CamScanner

30 | 2010 | SATURDAY • JANUARY

Singly linked list

(1) Traversal

Step 1 → PTR = FIRST
2. Repeat step (3) to (4) while PTR != NULL
3. Print info (PTR)
4. PTR = LINK (PTR)
5. STOP.

(2) Insertion in a singly linked list

(a) Insertion at Beginning

1. If AVAIL = NULL then write Linked list is overflow and stop.
2. PTR = AVAIL
   AVAIL = LINK (AVAIL)
   Read info (PTR)
3. LINK (PTR) = FIRST
   FIRST = PTR
4. STOP

31 SUNDAY

THINGS TO DO

01 | 2010 | FEBRUARY • MONDAY

(b) Insertion at End

Step 1 → If AVAIL = NULL then write overflow and stop.
2. PTR = AVAIL
   AVAIL = LINK (AVAIL)
   Read info (PTR)
3. CPT = FIRST
4. Repeat step (5) while LINK (CPT) != NULL
5. CPT = LINK (CPT)
6. LINK (CPT) = PTR
   LINK (PTR) = NULL
7. STOP

(c) Insertion after given INFO node

Step 1. If AVAIL = NULL then write overflow and stop.
2. Read data as information of node after which insertion will made.
3. PTR = AVAIL
   AVAIL = LINK (AVAIL)
   Read info (PTR)

THINGS TO DO

Scanned by CamScanner

02 2010 TUESDAY • FEBRUARY

step4 CPT = FIRST

5. Repeat step(6) while Info(CPT) != Data

6. CPT = Link(CPT)

7. LINK(PTR) = Link(CPT)
   LINK(CPT) = PTR

8. stop.

(d) Insertion in sorted list

1. If AVAIL = NULL then write overflow and stop.

2. PTR = AVAIL
   AVAIL = LINK(AVAIL)
   Read Info(PTR)

3. CPT = FIRST

4. Repeat step(5) while Info(CPT) < Info(PTR)

5. TPT = CPT
   CPT = Link(CPT)

6. Link(TPT) = PTR
   Link(PTR) = CPT

7. stop.

THINGS TO DO

03 2010 FEBRUARY • WEDNESDAY

Program for creation of Linked list

```
#include <stdio.h>
#include <conio.h>
#include <alloc.h>

struct node
   { int info;
     struct node *link;
   };

struct node * first;

void main()
  {
     void create();
     clrscr();
     create();
     getch();
  }

void create()
  {
     struct node *ptr, *cpt;
     char ch;

ptr = (struct node *) malloc(sizeof(
                    struct node));
```

THINGS TO DO

Scanned by CamScanner

```
printf (" Input first node info");
scanf("%d", &ptr->info);
first = Ptr;
do
{
cpt = (struct node*) malloc (sizeof(
        struct node));
printf (" enter next node info ");
scanf("%d", &(cpt->info));
ptr -> link = cpt;
Ptr = cpt;
printf("Press (Y/N) for more node");
scanf("%c", &ch);
}
while (ch =='Y' || ch == 'y');
ptr -> link = NULL;
}
```

THINGS TO DO

Deletion in singly linked list

(a) Deletion from Begining

1. If FIRST = NULL then write Underflow and stop.
2. PTR = FIRST
   FIRST = LINK (PTR)
3. LINK (PTR) = AVAIL
   AVAIL = PTR
4. stop.

(b) Deletion from End

1. If first = NULL then write underflow and stop.
2. PTR = first
3. Repeat step (4) while LINK (PTR) != NULL
4. CPT = PTR
   PTR = LINK (PTR)
5. LINK (CPT) = NULL
6. LINK (PTR) = Avail
   Avail = PTR
7. stop.

THINGS TO DO

Scanned by CamScanner

06 2010 SATURDAY • FEBRUARY

(c) Deletion of given node

1. If first = NULL then write underflow and stop.
2. Read data as info of node to be deleted.
3. Ptr = first
4. Repeat step (5) while info (Ptr) ≠ data
5. cpt = Ptr
   Ptr = Link (Ptr)
6. Link (cpt) = Link (Ptr)
7. Link (Ptr) = Avail
   Avail = Ptr
8. stop.

7 SUNDAY

THINGS TO DO

08 2010 FEBRUARY • MONDAY

Circular Linked List

(1) Traversal

(1) PTR ← First
    Process info (Ptr)
    Ptr ← Link (Ptr)
(2) Repeat step 3 while Ptr ≠ First
(3) Process info (Ptr)
(4) Ptr ← Link (Ptr)
(5) Stop.

(2) Insertion

(a) At begin

1. If Avail = NULL then write "Overflow" and stop.
2. Ptr ← Avail
   Avail ← Link (Avail)
   Read info (Ptr)
3. cpt ← First
4. Repeat step 5 while Link (cpt) != First
5. cpt ← Link (cPT)
6. Link (Ptr) ← First
   First ← Ptr
   Link (cpt) ← First

THINGS TO DO

Scanned by CamScanner

09 2010 TUESDAY • FEBRUARY

7. Stop.

(b) At end

1. If avail = NULL then write "overflow" and stop.
2. Ptr ← Avail
   Avail ← link (Avail)
   Read Info (Ptr)
3. cPt ← First
4. Repeat step 5 while link(cPt) != First
5. cPt ← link(cPt)
6. link (cPt) ← Ptr
7. link (Ptr) ← First
8. Stop.

3. Deletion

(a) From Begin

1. If First = NULL then write "Underflow" and stop.
2. cPt ← First
3. Repeat step 4 while link (cPt) != First

THINGS TO DO

10 2010 FEBRUARY • WEDNESDAY

4. cPt ← link (cPt)
5. Ptr ← First
   First ← link (Ptr)
   link (cPt) ← First
6. link (Ptr) ← Avail
   Avail ← Ptr
7. Stop.

(b) From end

1. If First = NULL then write Underflow and stop.
2. cPt ← First
3. Repeat step 4 while link (cPt) != First
4. Ptr ← cPt
   cPt ← link (cPt)
5. link (Ptr) ← First
6. link (cPt) ← Avail
   Avail ← cPt
7. Stop.

THINGS TO DO

Scanned by CamScanner

11 2010 THURSDAY • FEBRUARY

Doubly linked list

(1) Traversing

(a) Forward Traversing

1. Ptr ← First
2. Repeat step 3 to 4 while Ptr != NULL
3. Process Info(Ptr)
4. Ptr ← RPT(Ptr)
5. stop.

(b) Backward Traversing

1. Ptr ← First
2. Repeat step 3 while RPt(Ptr) != NULL
3. Ptr ← RPt(Ptr)
4. Repeat step 5 to 6 while Ptr != NULL
5. Process Info(Ptr)
6. Ptr ← LPt(Ptr)
7. Stop.

THINGS TO DO

12 2010 Wk 07 DAY 043-322 FEBRUARY • FRIDAY

(2) Insertion

(a) At begin

1. If avail = NULL then write "overflow" and stop.
2. Ptr ← Avail
   Avail ← RPt(Avail)
   Read Info(Ptr)
3. Rpt(Ptr) ← First
   LPt(First) ← Ptr
   LPt(Ptr) ← NULL
   First ← Ptr
4. Stop.

(b) At End

1. If avail = NULL Write "overflow" and stop.
2. Ptr ← Avail
   Avail ← Rpt(Avail)
   Read Info(Ptr)
3. cPt ← First
4. Repeat step 5 while RPt(cPt) ≠ NUL

THINGS TO DO

Scanned by CamScanner

13 2010 SATURDAY • FEBRUARY

5 CPt ← RPt (CPt)

6. RPt (CPt) ← Ptr
LPt (Ptr) ← CPt
RPt (Ptr) ← NULL

7. stop.

(c) After given info of node

1. if avail = NULL then write "Overflow" and stop.

2. Ptr ← Avail
Avail ← RPt (Avail)
Read Info (Ptr)

3. Read data

4 CPt ← First

5. Repeat step 6 while Info (CPt) ≠ data

14 SUNDAY 6. CPt ← RPt (CPt)

THINGS TO DO 7. TPt ← RPT (CPT)
RPt (CPt) ← Ptr
LPt (Ptr) ← CPt
RPt (Ptr) ← TPt
LPt (TPt) ← Ptr

8. stop.

15 2010 FEBRUARY • MONDAY

(d) Insertion in sorted List

1 If avail = NULL then "Overflow" and stop

2. Ptr ← Avail
Avail ← RPt (Avail)
Read Info (Ptr).

3. CPt ← First

4 Repeat step 5 to 6 while Info (CPt) < Info (Ptr)

5. CPt ← RPt (CPt)

6. TPt ← LPt (CPt)

7 RPt (TPt) ← Ptr
LPt (Ptr) ← TPt
RPt (Ptr) ← CPt
LPt (CPt) ← Ptr

8. stop.

THINGS TO DO

Scanned by CamScanner

16 2010 TUESDAY • FEBRUARY

(3) Deletion

from Begin

1 If first = NULL then write underflow and stop.

2. Ptr = first
first = RPT(Ptr)
LPt(first) = NULL

3. RPt(Ptr) = Avail
Avail = Ptr

4 stop

from End

1 If first = NULL then write Underflow and stop.

2. Ptr = first

3. Repeat step 4 while RPt(Ptr) ≠ NULL

4. Ptr = RPt(Ptr)

5. cPt = LPt(Ptr)
RPt(cPt) = NULL

6. RPt(Ptr) = Avail
Avail = Ptr

7 Stop.

THINGS TO DO

17 2010 FEBRUARY • WEDNESDAY

Deletion of Particular node

1. If first = NULL then write underflow and stop.

2. Read data as Info.

3. Ptr = first

4. Repeat step 5 while Info(Ptr) ≠ Data

5. Ptr = RPt(Ptr)

6. cPt = LPt(Ptr)
TPt = RPT(Ptr)
RPt(cPt) = TPt
LPt(TPt) = cPt

7. RPt(Ptr) = Avail
Avail = Ptr

8. stop.

Doubly circular linked list

Insertion at begin

1 If Avail = NULL then write overflow and stop.

2. Ptr = Avail
Avail = RPt(avail)
Avail = LPt(avail)
Read Info(Ptr)

THINGS TO DO

Scanned by CamScanner

3. Tpt = Lpt (first)
4 Rpt (PtR) = first
Lpt (first) = Ptr
5. LPt (Ptr) = TPt
RPt (TPt) = Ptr
6. first = Ptr
7. stop.

At End

1. If avail = NULL then write "Overfl ow" and stop.
2. Ptr = Avail
Avail = LPt (Avail)
Avail = Rpt (avail)
Read Info (Ptr)
3. TPt = LPt (first)
4 RPt (TPt) = Ptr
LPt (Ptr) = TPt
5. RPt (Ptr) = first
LPt (first) = Ptr
stop.

Deletion from begin

1. If first = NULL then write "Under flow" and stop.
2. TPt = first
Ptr = RPt (first)
CPt = LPt (first)
3. LPt (Ptr) = CPt
RPt (CPt) = Ptr
4. first = Ptr
5. LPt (TPt) = Anail
RPt (TPt) = Anail
Anail = TPt
6 stop.

Deletion from End

1. If first = NULL then write "Under flow" and stop
2. Ptr = LPt (first)
CPt = LPt (Ptr)
3 - LPt (first) = CPt
RPt (CPt) = first

Scanned by CamScanner

20 | 2010
DAY 051-314 Wk 08
SATURDAY • FEBRUARY

4- LPt (Pt+1) = Avail
RPt (Pt+1) = Avail
Avail = Pt+1

5 - stop.

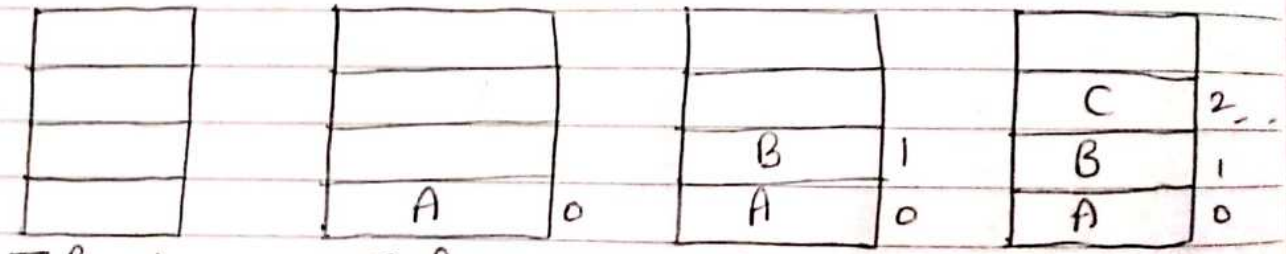
~~Doubly / circular~~ ~~Linked list~~

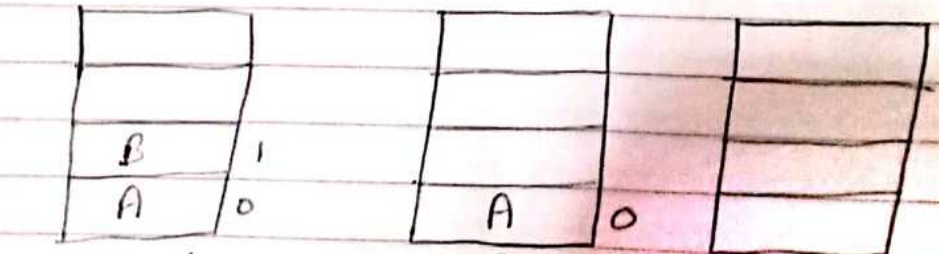
Scanned by CamScanner

25 | 2010
DAY 056 309 WK 09
THURSDAY • FEBRUARY

stacks

A stack is an ordered collection of homogeneous data elements where the insertion and deletion operations occur at only one end. The end is known as top of stack. The last element inserted will be on top of stack and deletion is done from the same end. So it is called the LIFO (Last In first out).





operation of stack

THINGS TO DO

Push and Pop The process of adding a new element to the top of stack is

Scanned by CamScanner

called Push operation. If stack is full and we want to insert more element, this situation is called overflow.

The process of Deleting an element from top of stack is called the Pop operation. When stack is empty and we want to delete more element, this situation is called the underflow.

Algorithm for Push & Pop

Push

1. If Top = Max - 1 then write overflow and stop.
2. Read data
3. Top = top + 1
4. stack [Top] = data
5. stop.

Pop

1. If top < 0 then write underflow and stop.
2. stack [Top] = NULL
3. Top = Top - 1
4. stop.

THINGS TO DO

Scanned by CamScanner

27 | 2010
DAY 058-307 WK 09
SATURDAY • FEBRUARY

FEBRUARY
S M T W T F S
1 2 3 4
7 8 9 10 11
14 15 16 17 18
21 22 23 24 25
28

Application of stacks

(a) checking the validity of an airthmetic expression

[(A+B) - {C+D}] - [F+G]

	symbol scanned	stack
(1)	[	[
2.	(	[(
3.	A	[(
4.	+	[(
5.	B	[(
6.	)	[
7.	-	[
8.	{	[{
9.	C	[{
10.	+	[{
11.	D	[{
12.	}	[
13.	]	-
14.	-	-
15.	[	[
16.	F	[
17.	+	[
18.	G	[
19.	]	-

Expression is valid.

28 SUNDAY
THINGS TO DO

Scanned by CamScanner

(b)(i) Infix to Postfix conversion

Polish notation

Transforming an exp. from one form to another is called the polish notation. An airthmetic exp. can be represented in various form such as Prefix, Infix and Postfix.

The Pre, In and Post refer to the relative position of the operator with respect to its operands. If the opr. is placed before its 2 operands the exp. is called the prefix, If the opr. is placed in middle it is known as Infix and if opr. is placed after the 2 operands it is known as Postfix.

Prefix	+ A B
Infix	A + B
Postfix	A B +

Priority of operators

Highest exponential ($\wedge$ or $\uparrow$)
multiplication (*, or x)
division (/ or ÷)

Lowest Addition +
subtraction −

THINGS TO DO

Scanned by CamScanner

02 | 2010
DAY 061-304 Wk 10
TUESDAY • MARCH

Ex (1) (A+B) / (C-D)

(AB+) / (CD-)

X Y

X / Y

XY/

Put the value of X and Y

AB+CD-/

(2) A*B + C/D

~~AAAAA~~

(A*B) + (C/D)

(AB*) + (CD/)

X Y

X + Y

XY+ Put the values of X and Y

AB*CD/+

(3) A + (B*C - (D/E ^ F)*G)*H

THINGS TO DO

Result **ABC*DEF^/G*-H*+**

Scanned by CamScanner

(11) Infix to Postfix using stack

Algorithm

1. Scan the infix exp. from left to right.
2. If scan char. is operand then that will display into postfix exp.
3. If char. is operator then push into stack.
4. If scan operator is greater than stacked operator then scanned operator will also push into stack.
5. If scan opr. is less than or equal to stacked operator then stacked opr. will pop out from stack to postfix exp. and scanned opr. will push into stack.
6. If scanned char. is right parenthesis then till left parenthesis of stack, all opr. will pop out to postfix exp. and left will cancelled by right parenthesis.
7. Repeat step 1 to 6 until end of exp.

Ex (1)

5 * (6 + 2) - (12/4)

	symbol scanned	stack	postfix exp.
THINGS TO DO! 1.	5		5
2.	*	*	5
3.	(	*, (	5
4.	6	*, (	5, 6
5.	+	*, (, +	5, 6

Scanned by CamScanner

6.	2	*, (, +	5, 6, 2
7.	)	*	5, 6, 2, +
8.	-	-	5, 6, 2, +, *
9.	(	-, (	5, 6, 2, +, *
10.	12	-, (	5, 6, 2, +, *, 12
11.	/	-, (, /	5, 6, 2, +, *, 12
12.	4	-, (, /	5, 6, 2, +, *, 12, 4
13.	)	)	5, 6, 2, +, *, 12, 4, /
14.			5, 6, 2, +, *, 12, 4, /

ex (2) a + b * c + (d * e + f) * g

Result | a b c * + d e * f + g * +

(c) Evaluation of Postfix Exp. using stack

Algorithm

1. If an operand is encountered, Push it on stack.
2. If an opr. 'op' is encountered
   (i) Pop 2 elements of stack, where A is the top element and B is the next top element.
   (ii) Evaluate B op A
   (iii) Push Result on stack.
3. The Evaluated value is equal to the value at the top of stack.

THINGS TO DO

Scanned by CamScanner

Ex — 5, 6, 2, +, *, 12, 4, /, -

	symbol scanned	stack	operation B op A
1	5	5	
2	6	5, 6	
3	2	5, 6, 2	
4	+	5, 8	[6 + 2] (A = 2, B = 6)
5	*	40	[5 * 8] (A = 8, B = 5)
6	12	40, 12	
7	4	40, 12, 4	
8	/	40, 3	[12 / 4] (A = 4, B = 12)
9	-	37	[40 - 3] (A = 3, B = 40)

Q1 (1)

Infix to Prefix

A / B ^ C - D

A / (B ^ C) - D

A / (^ B C) - D

(^ B C) = X

A / X - D

/ A X - D

/ A X = Y

Y - D

- Y D

Put the values

- / A ^ B C D

Scanned by CamScanner

MARCH
S M T W T F S
1 2 3
7 8 9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30 31

(2)

$(A - B/C) * (D * E - F)$

$*-A/BC-*DEF$

Infix to Prefix Using stack

(1) Reverse the I/P string.
(2) Examine the next element in I/P.
(3) If it is operand add to o/p string
(4) If it is closing Parenthesis, Push it on stack.
(5) If it is an operator
(a) If stack is empty push opr. on stack
(b) If the top of stack is closing Parenthesis, Push opr. on stack.
(c) If it has same Priority or higher Priority then the top of stack, Push opr.
(d) else pop opr. from stack and add it to o/p string, repeat step 5.
(6) If it is a opening Parenthesis, pop opr. from stack and add to o/p string until a closing Parenthesis is encountered. Pop and discard the closing Parenthesis.
(7) If there is more I/P go to step 2.
(8) If there is no more I/P pop the remaining opr. and add to o/p str
(9) Reverse the o/p string.

7 SUNDAY

THINGS TO DO

Scanned by CamScanner

Ex

2 * 3 / (2 - 1) + 5 * (4 - 1)

Reverse string is →

) 1 - 4 (* 5 +) 1 - 2 (/ 3 * 2

char scanned	stack	Prefix Exp.
)	)	
1	)	1
-	) -	1
4	) -	14
(	empty	14 -
*	*	14 -
5	*	14 - 5
+	+	14 - 5 *
)	+)	14 - 5 *
1	+)	14 - 5 * 1
-	+) -	14 - 5 * 1
2	+) -	14 - 5 * 12
(	+	14 - 5 * 12 -
/	+ /	14 - 5 * 12 -
3	+ /	14 - 5 * 12 - 3
*	+ / *	14 - 5 * 12 - 3
2	+ / *	14 - 5 * 12 - 32
	empty	14 - 5 * 12 - 32 * / +

THINGS TO DO

Reverse the o/p.

+ / * 23 - 21 * 5 - 41

Scanned by CamScanner

09 2010

DAY 068-297 WK 11

TUESDAY • MARCH

ex 12, 6, 3, -, 1, 2, 1, 5, +, *, +

(a) convert into Infix

(b) and evaluate Infix Exp

Sol

(a) scan left to right

12, [6-3], /, 2, [1+5], *, +

[12/(6-3)], [2*(1+5)], +

(12/(6-3)) + (2*(1+5))

(b) (12/3) + (2*6)

4 + 12 = 16 Ans

THINGS TO DO

Scanned by CamScanner

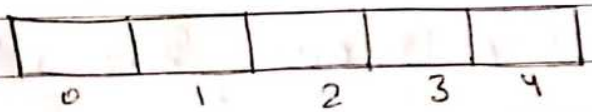
Queue

A Queue is a linear list of elements in which deletion can take place only at one end, called front and insertion can take place only at the other end called rear. Queue is called the FIFO (first in first out).

It is a homogeneous collection of elements. The 2 Basic operations that are possible in queue are →

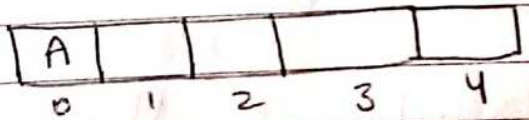
Insert an element to the queue (Push)
Deletion of element from queue (Pop).

Ex



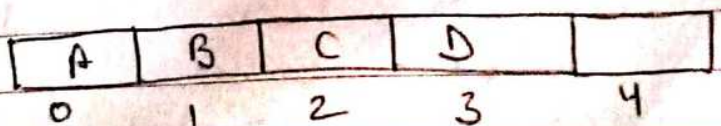
front = -1 empty queue
Rear = -1

Insert A



front = Rear = 0

Insert B C D



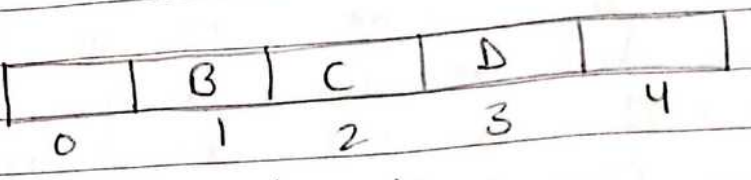
front = 0
Rear = 3

THINGS TO DO

Scanned by CamScanner

11 2010
DAY 070-295 WK 11
THURSDAY • MARCH

Delete of an Element



front = 1
Rear = 3

Insertion E



front = 1
Rear = 4

Algorithm for Queue Insertion

1. If rear = max then Write Queue is Overflow & stop.
2. Read data
3. If front = 0 then
   (a) front = front + 1
   (b) Rear = Rear + 1
   (c) Q (Rear) = Data
   else
   Rear = Rear + 1
   Q(Rear) = Data
4. Stop.

THINGS TO DO

Scanned by CamScanner

Deletion

1. If front = 0 then write Queue is Underflow & stop.
2. If front = Rear then
   - Q(front) = NULL
   - front = 0

   else
   - Q(front) = NULL
   - front = front + 1
3. stop.

condition

Overflow → If Rear = Max

Underflow → If front = 0

One element → If front = Rear.

Scanned by CamScanner

13 2010
DAY 072 293 WK 11
SATURDAY • MARCH

MARCH
S M T W
1 2 3
7 8 9 10
14 15 16 17
21 22 23 24
28 29 30 31

Algo. for checking the validity of an Arithmetic exp.

→ whenever an opening Parenthesis is encountered, it is pushed onto the stack.

→ whenever a closing Parenthesis is encountered, the stack is examined.

→ If the stack is empty, the closing Parenthesis does not have an opening parenthesis and the exp. therefore invalid.

→ If the stack is not empty, we pop the stack and check whether the popped item corresponds to the closing Parenthesis.

→ If a match occurs, we continue. Otherwise the exp. is invalid.

→ when the end of the exp. is reached the stack must be empty otherwise the exp. is invalid.

14 SUNDAY
THINGS TO DO

Scanned by CamScanner

Circular Queue

In circular Queue the elements Q[0], Q[1] - - Q[n-1] is represented in a circular fashion. A circular Queue is one in which the Insertion of a new element is done at the very first location of the Queue if the last loc. of Queue is full

Algo. of Insertion in circular Queue

step1 If (front = 0 && Rear = max-1) || (front = rear+1)
then write Queue is overflow and stop.

step2 Read data to Insert.

step3 If (front = -1)
(a) front = 0
(b) Rear = 0

step4 If (Rear = Max-1) then Rear = 0
else
Rear = Rear + 1

step5 coneue [Rear] = Data

step6 stop.

Scanned by CamScanner

Delete operation on Queue

step1 if (front = -1) then write queue is underflow and stop.

step2 if front = Rear then
(a) Queue [Front] = NULL
(b) front = -1
(c) Rear = -1

step3 if front = Max - 1
(a) Queue [Front] = NULL
(b) front = 0
Else
(a) Queue [front] = NULL
front = front + 1

step4 stop

THINGS TO DO

Scanned by CamScanner

Ex

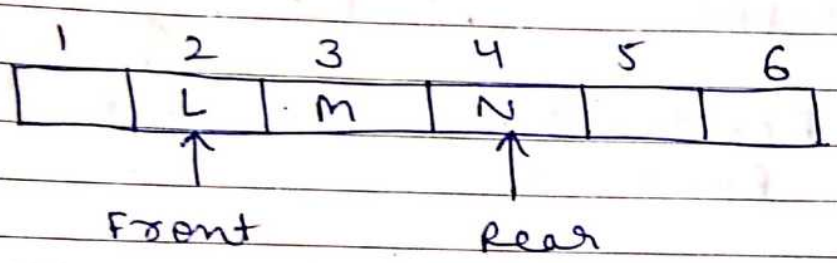
A circular Queue capable of accommodating maximum 6 elements.

Rear = 4, front = 2

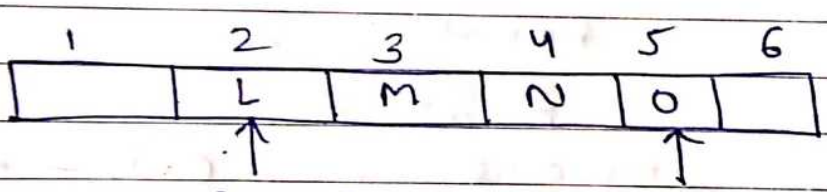
Queue = , L, M, N, — —

(a) Add O (b) Add P (c) Delete 2 Letter
(d) Add Q, R, S (e) Delete one letter.

Sol.

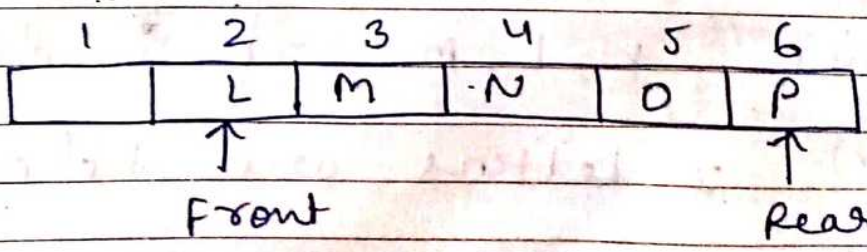


(i) Add O



Front = 2
Rear = 5

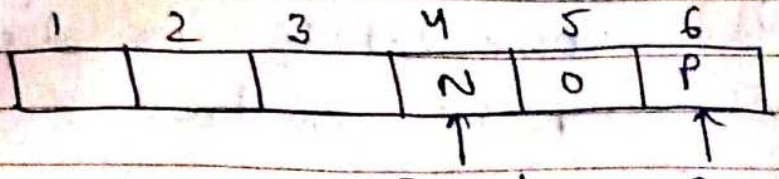
(ii) Add P



front = 2
Rear = 6

THINGS TO DO

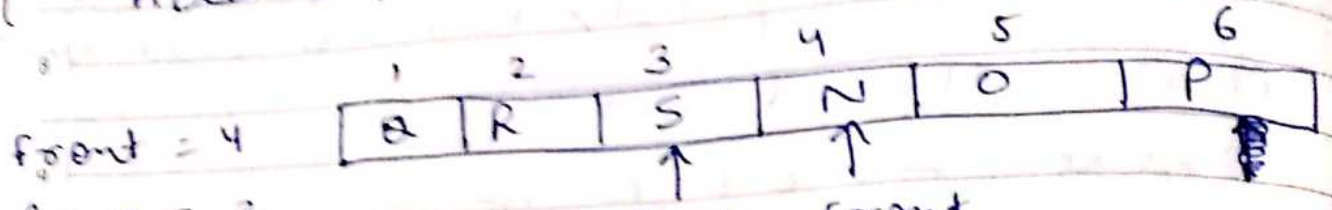
(iii) Delete 2 letters



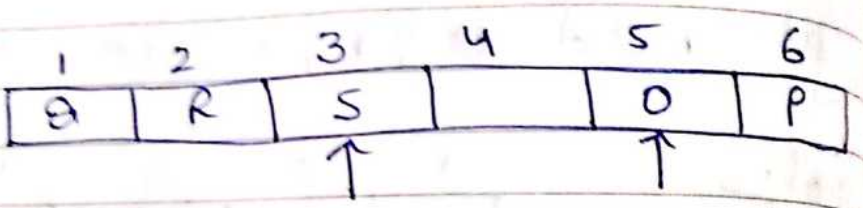
front = 4
Rear = 6

Scanned by CamScanner

(IV) Add O, R, S



(V) Delete one letter



Ex2 Queue of 6 memory cells.

Front = 2

Rear = 4

Queue - A C D - -

(i) F is added

(ii) 2 letters are deleted

(iii) K, L, M are added

(iv) 2 letters are deleted

(v) S is added

THINGS TO DO

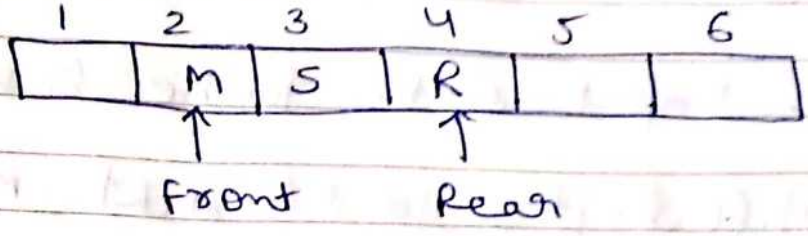
(VI) One letter is deleted

(VII) R is added

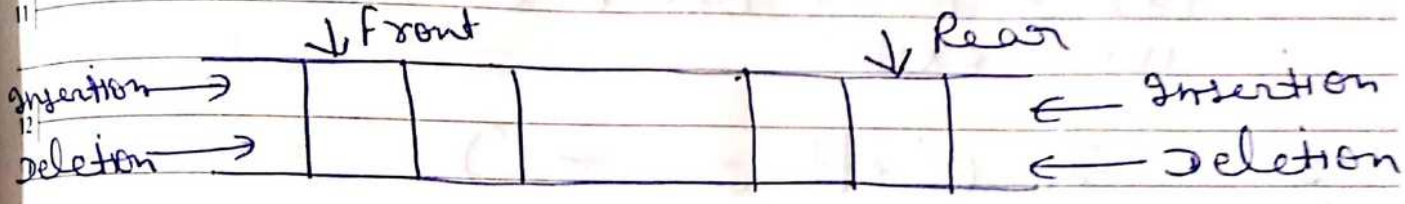
(VIII) One letter is deleted.

Scanned by CamScanner

final



DEQUEUE

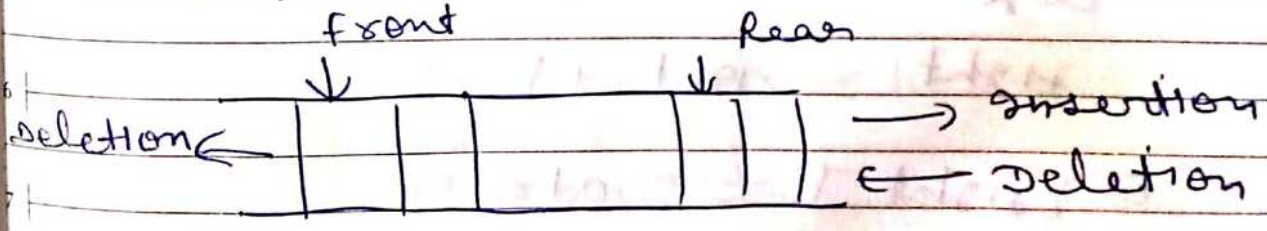


Representation of DEQUEUE

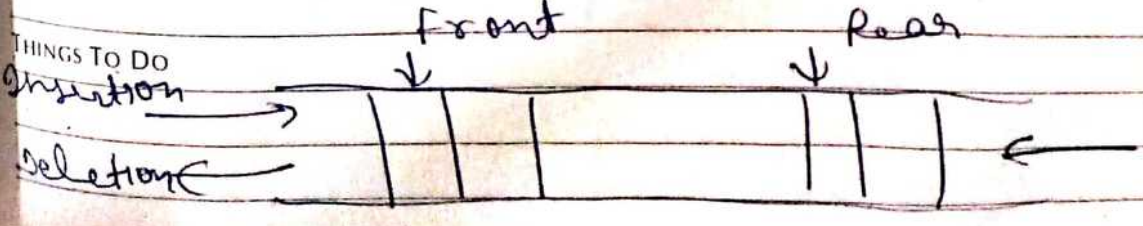
(a) Using a Doubly linked list
(b) Using circular Array.

Types of Dequeue

(a) Input Restricted



(b) Output Restricted



THINGS TO DO

Scanned by CamScanner

Insert element at Right side of Deq

Step1 Input data to be inserted

2 If ((left == 0 && right = max-1) ||
(left == right + 1))
(a) Display Queue Overflow
(b) Exit

3 If (left == -1)
left = 0
Right = 0

4 Else
If (right == max-1)
right = 0
else
right = right + 1

5 Q[right] = Data

6 Stop

Scanned by CamScanner

Insert Element at left side of Deque

1. Input data to be inserted.
2. If ((left == 0 && right == max-1) || (left == right + 1))
   (a) Display Queue overflow
   (b) stop.
3. If (left == -1)
   left = 0
   Right = 0
4. else
   If (left == 0)
   left = Max-1
   else
   left = left - 1
5. D[left] = Data
6. Exit

THINGS TO DO

Scanned by CamScanner

Delete an element from Right end

1. if (left == -1)
   Display underflow
   stop
2. Data = D[right]
3. if (left == right)
   left = -1
   right = -1
   else
   if (right == 0)
   right = max - 1
   Else
   right = right - 1
4. Exit

Scanned by CamScanner

Delete an element from left side

1. if (left == -1)
   Display underflow
   Stop.
2. data = a[left]
3. if (left == right)
   left = -1
   right = -1
4. else
   if (left == max - 1)
   left = 0
   else
   left = left + 1
5. Exit

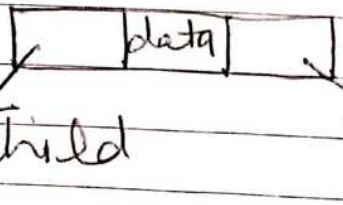
Scanned by CamScanner

Trees

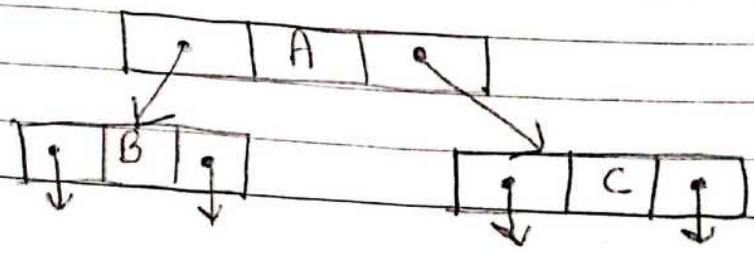
A tree is either empty, or it consist of a node called root together with 2 binary trees called the left subtree and right subtree of the root. The defination of a tree does not impose any combination on the no. of children of a given node. This no. can vary from 0 to any integer.

Terminology Used

1. **Node** We will use the term node rather than vertex with tree. This is the main component of any tree structure. It stores the Actual data along with links to other nodes.



2. **Parent** The Parent of a node is the immediate predecessor of that node.



THINGS TO DO

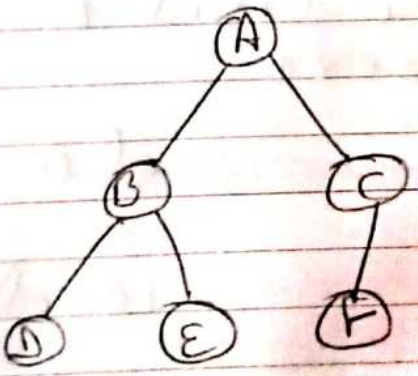
Scanned by CamScanner

A is the Parent of node B and C.

3. Child The Immediate successor of a node are called child nodes. A child which is placed at left side is called left child and which is placed at right side is called the right child.

B and C are the left and right child of A.

4. Root It is node which has no Parent. There can be only one root in tree.



A is the root.

5. Leaf Node (External node) A node which has no child is called leaf node. D, E, F are leaf node.

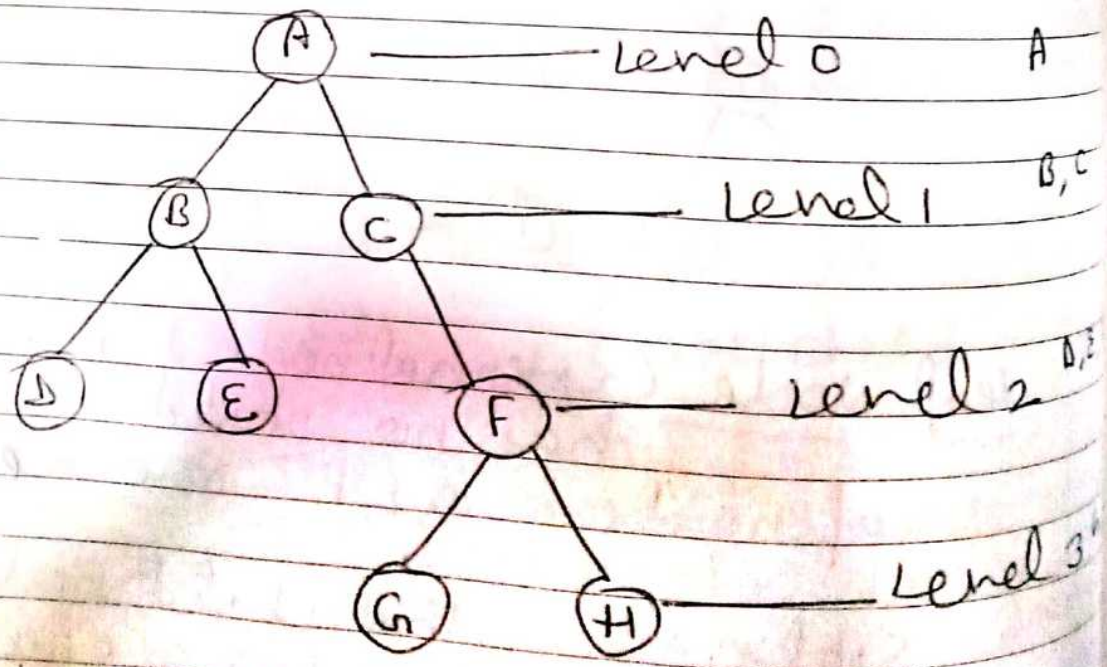
THINGS TO DO 6. Siblings The child node of a given Parent node are called the siblings. D and E are siblings.

Scanned by CamScanner

7. Branch or Edge connecting b/w 2 nodes. A node can have more than one branch. In fig. A has 2 branches.

8. Path Each node has to be reachable from root through a unique sequence of edges called a path. The no. of edges in a path is called length of path.

9. Level of a node The level of a node is the no. of edges along the unique path b/w it and the root. The level of root is defined as zero.



10. Height The Height of a tree is the max level of a node in the tree.

Scanned by CamScanner

$h = I_{max} + 1$

I is the level.

Height of empty tree is zero and height of tree containing 1 node is one.

Binary tree A tree is a binary tree if and only if -

(1) It has a root node, which may not have any child node (0 child, NULL Tree).
2. Root node may have 1 or 2 child nodes. Each such node forms a binary tree itself
3. No. of child nodes can be 0, 1, 2 not more than 2.
4. There is a unique path from the root to every other node.

Properties of binary tree -

(1) The no. of external nodes is 1 more than the no. of internal node.

(2) The no. of external nodes is atleast $h+1$ and at most 2^h where h is the height of tree.

(3) The no. of internal nodes is atleast h and atmost $2^h - 1$.

(4) The total no. of nodes in a binary tree is atleast $2^h + 1$ and at most $2^{h+1} - 1$.

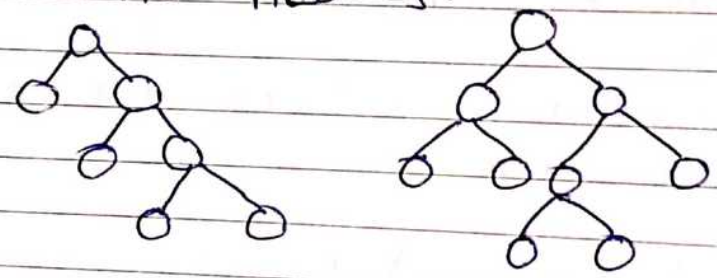
Scanned by CamScanner

(5) The height h of a binary tree with n nodes is atleast log n+1 and atmost n.

(6) A binary tree with n nodes has exactly n-1 edges.

Types of binary tree

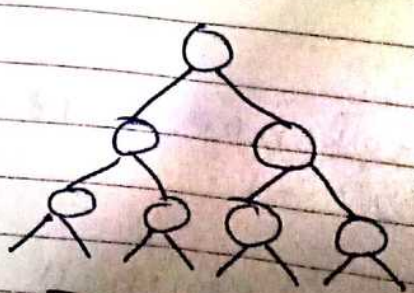
(1) **strictly binary tree** A binary tree is a strictly binary tree if and only if each node has exactly 2 child node or no nodes.



(2) **Full Binary tree** A binary tree is a Full binary tree if and only if -

(1) each non leaf node has exactly 2 child node.

(2) All leaf node are at same lev



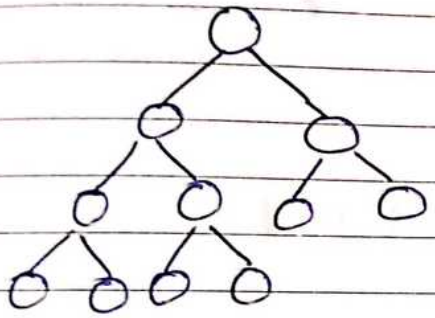
THINGS TO DO

Scanned by CamScanner

(3) Complete Binary tree A binary tree is a complete binary tree if and only if —

(a) Level 0 to h-1 represent a full binary tree of height h-1.

(b) one or more nodes in level h-1 may have 0 or 1 child node.



(4) Extended binary tree A binary tree is called extended binary tree if every node of tree has zero or 2 children. It is also known as 2-tree.

A binary tree can be converted to an EBT by adding new nodes to its leaf node, and to the nodes which have only one child. The nodes of original tree are called internal nodes and the new nodes that are added to binary tree, are called external nodes. It is very useful to represent algebric expression.

THINGS TO DO

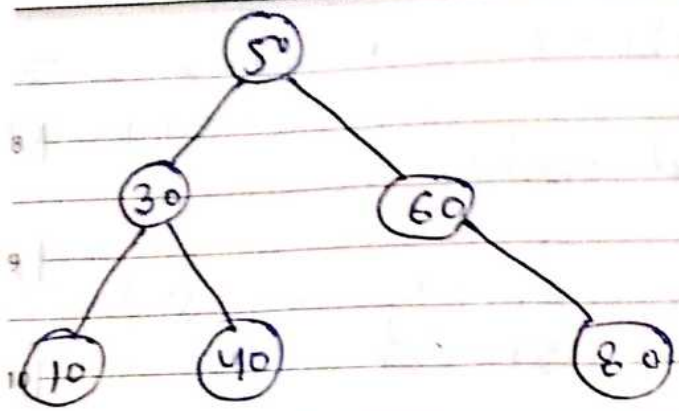
Scanned by CamScanner

01

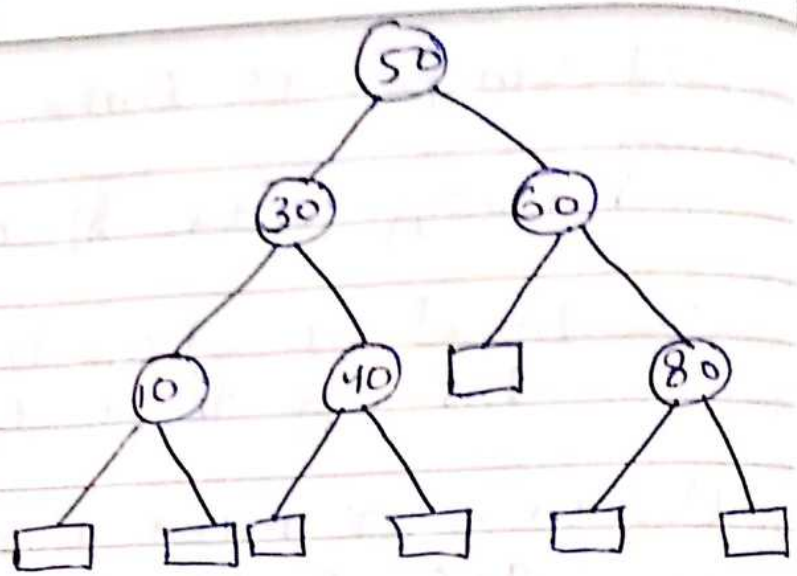
2010

DAY 091-274 WK 14

THURSDAY • APRIL



Binary tree



EBT

Scanned by CamScanner

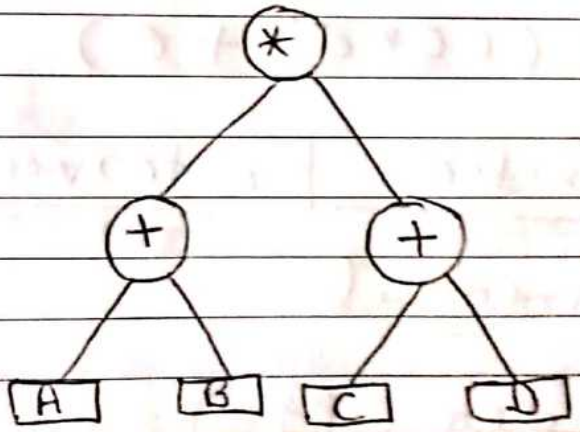
Expression tree

The arithmetic expression represented as binary tree are known as expression tree.

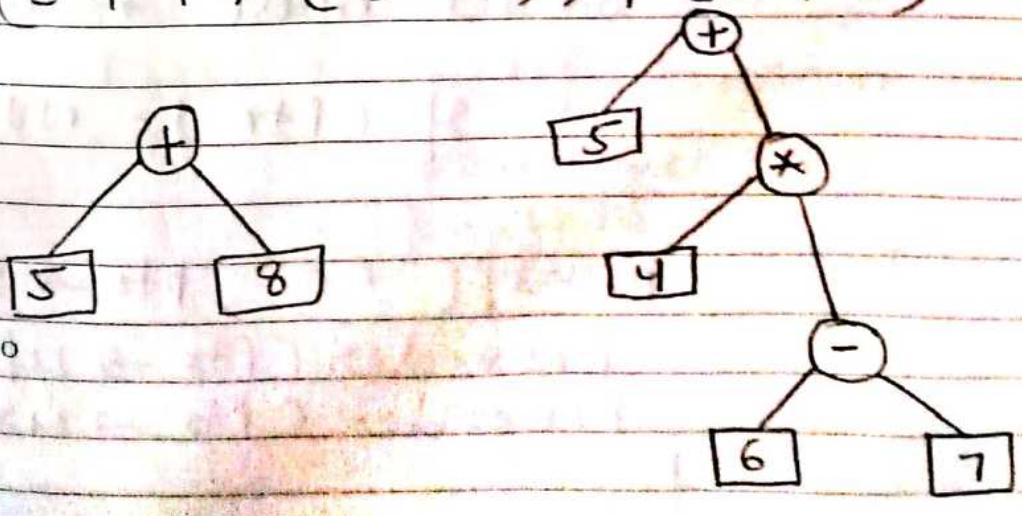
(A + B) * (C + D)

operator ○

operand □



(5 + 4 * (6 - 7)) / (5 + 8)

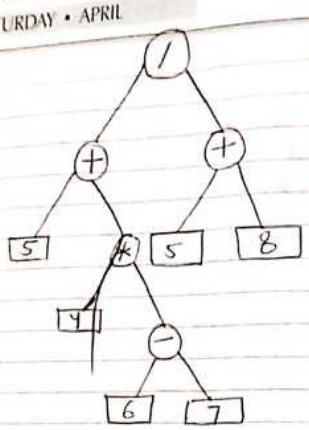


THINGS TO DO

Scanned by CamScanner

03

SATURDAY • APRIL



Ex (a - b) / ((c*d) + e)

Recursive function for traversal

Preorder traversal

```
Preorder (Ptr)
    {   struct node *ptr;
        {   if (ptr != NULL)
        {
        printf("%c", ptr -> info);
        Preorder (Ptr -> Left);
        Preorder (Ptr -> Right);
        }
        }
    }
```

4 SUNDAY

THINGS TO DO

2010

APRIL • MONDAY

05

Inorder

```
inorder (ptr)
    {
        struct node *ptr;
        {
            if (ptr != NULL)
            {
                inorder (ptr -> Left);
                printf("%c", ptr -> info);
                inorder (ptr -> Right);
            }
        }
    }
```

Postorder

```
Postorder (Ptr)
    {   struct node *ptr;
        {   if (ptr != NULL)
        {
        postorder(ptr -> Left);
        postorder (ptr -> Right);
        printf("%c", ptr -> info);
        }
        }
    }
```

THINGS TO DO

Scanned by CamScanner

Non Recursive function for traversal

Preorder

Initially ptr contains the address of root.

1. Push the address of root on stack.
2. Pop an address from stack
3. If Pop address is not NULL
   traverse the node
   Push right child of node on stack
   Push left child of node on stack
4. Repeat step 2, 3, until stack is not empty.

Inorder

1. Repeat step 2 and 3 while stack is not empty or Ptr is ≠ NULL
2. If Ptr ≠ NULL
   (a) Push Ptr on stack
   (b) Ptr = left (Ptr)
3. If Ptr = NULL

   (a) Pop the element from stack.
   (b) Traverse the node
   (c) Ptr = Right (Ptr)

Post order

1. Initially Ptr has the address of root.
2. Repeat step 3 to 5 while (Ptr != NULL
3. Top = top + 1
   stack [top] = Ptr
4. If (Right (Ptr) != NULL)
   {
   top = top + 1
   stack [top] = Right (Ptr)
   }
5. Ptr = Left (Ptr)
6. Pop from stack.
7. while (Ptr > 0)
   { traverse Ptr
   Ptr = stack [top]
   top = top - 1
   }
8. If ptr < 0 then

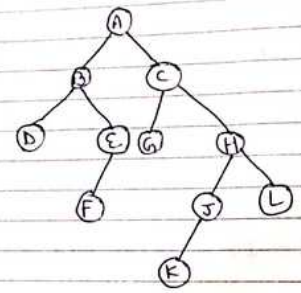
Scanned by CamScanner

08

THURSDAY • APRIL

{ ptr = Right (ptr)
goto step 2
}
9 step

Ex



Preorder A B D E F C G H J K L

Postorder D F E B G K J L H C A

Inorder D B F E A G C K J H L

Creation of Binary tree

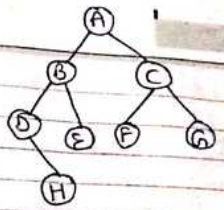
Ex1 Preorder A B D H E C F G

Things To Do Inorder D H B E A F C G

2010

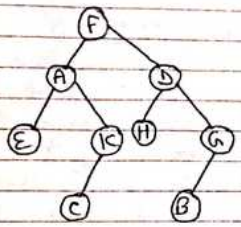
APRIL • FRIDAY

09



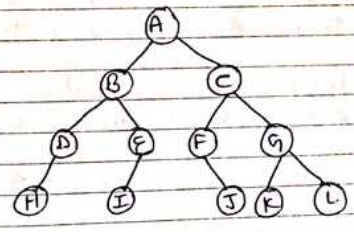
Ex2 Pre F A E K C D H G B

In E A C K F H D B G



Ex3 Post H D I E B J F K L G C A

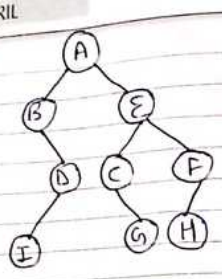
In H D B I E A F J C K G L



Things To Do 4 Post 9 D B G G C H F E A

In B I D A C G E H F

Scanned by CamScanner



Threaded Binary tree

consider the linked representation of any binary tree, we notice that half of the entries in the pointer field LEFT and RIGHT will contains NULL. This space may be more efficiently used by replacing a NULL entries by special pointer called Threads, which points to the nodes higher in the trees. Such trees are called Threaded binary trees.

[A binary tree with n nodes when n≥0 has exactly n+1 null links.]

11 SUNDAY

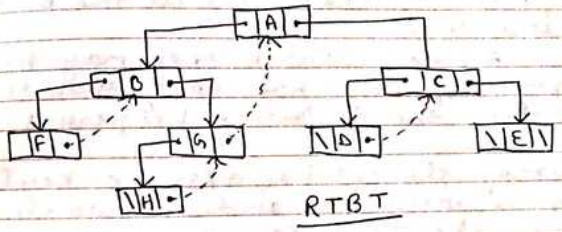
Types of threaded binary tree :-

THINGS TO DO

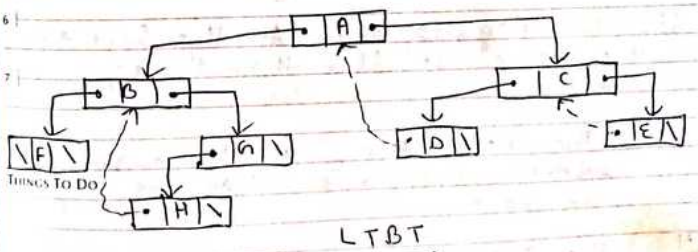
(1) The right NULL pointer of each node can be replaced by a thread to successor of that node under in-order traversal called a right threaded, and the tree will called a right threaded tree.

(2) The left NULL pointer of each node can be replaced by a thread to the predecessor of that node under in-order traversal called a left threaded and the tree will called a left threaded tree.

(3) Both left & Right NULL Pointers can be used to Point to Predecessor & successor of that node, respectively under in-order traversal. Such a tree is called a fully threaded tree.



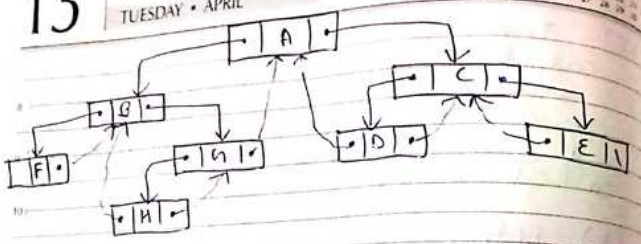
RTBT



THINGS TO DO

LTBT

Scanned by CamScanner



Binary search tree

A binary search tree is a binary tree. It is used to make the search operation faster. It is also called the Binary sorted tree.

A binary search tree may be empty. If it is not empty then it satisfies the following properties:-

(1) Every element has a key i.e. key(x) for element (x) and no two elements have the same key.

(2) The keys (if any) in the left subtree are smaller than the key in root.

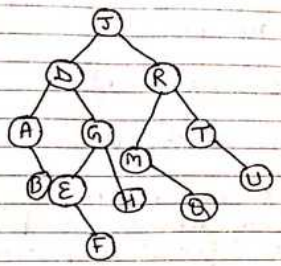
(3) The keys (if any) in the right subtree are larger than the key in root.

(4) The left and right subtrees are also binary search trees.

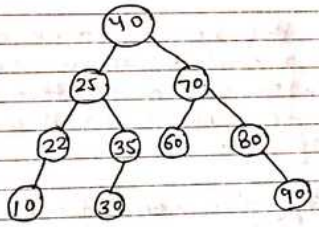
THINGS TO DO

Creation of BST

Ex1 J A D T G E A M H F B U B



Ex2 40 25 70 22 35 60 80 90 10 30



(a) Delete node 30
(b) " " 80
(c) " " 40

THINGS TO DO

Scanned by CamScanner

15 2010 THURSDAY • APRIL

Delete of a node Deletion of a node from a BST depends on its children. There are 3 cases:-

(a) z has no child.

(b) z has one child

(c) z has 2 child.

Case 1 If the node to be deleted z is a leaf node, simply make the pointer field of parent node of z to NULL.

Case 2 If the node to be deleted z has a child then simply set the pointer field of the parent node of z to point to the only child node of z.

Case 3 If the node to be deleted z has 2 children then find the in-order successor of the node to be deleted z. Place it in the node z. Then delete the inorder successor node similar to case 1 or case 2.

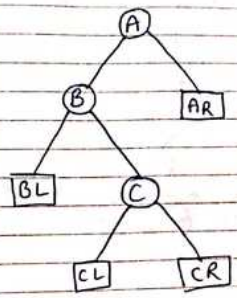
THINGS TO DO

2010 Wk 16 DAY 106-259 APRIL • FRIDAY 16

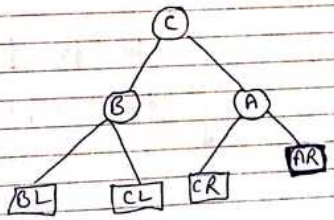
AVL Tree

Adelson Velskii & Landis

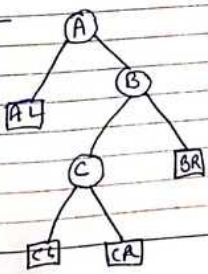
L-R



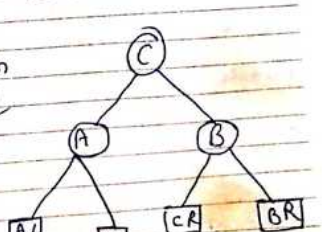
After L-R



R-L



After R-L



THINGS TO DO

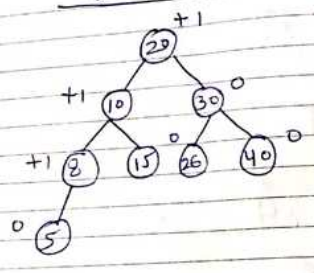
Scanned by CamScanner

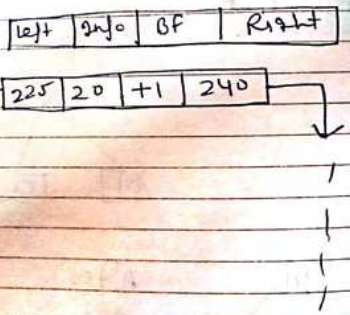
17 - 2010
SATURDAY • APRIL

Balance factor

Height of Left subtree - Height of Right subtree

-1, 0, +1



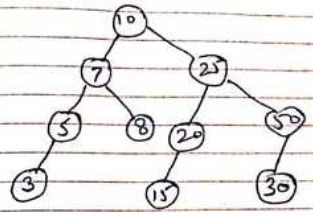


18 SUNDAY

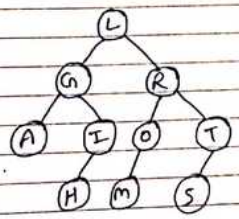
THINGS TO DO

2010
Wk 17 DAY 109-256
APRIL • MONDAY
19

Ex: 50, 25, 10, 5, 7, 3, 30, 20, 8, 15



Ex: ALGORITHMS



THINGS TO DO

Scanned by CamScanner

Graph

Graph is a non linear DS. Graph are DS which have wide range applications in real life like Airlines, analysis of electrical ~~circ~~ circuits, source, destination network, finding shortest routes etc.

A Graph G is a collection of 2 ~~sets~~ V and E where V is the vertices v_1, v_2, v_3 -- and E is the collection of edges e_1, e_2, e_3 -- etc. This can be represented as →

$$G = (V, E)$$

where

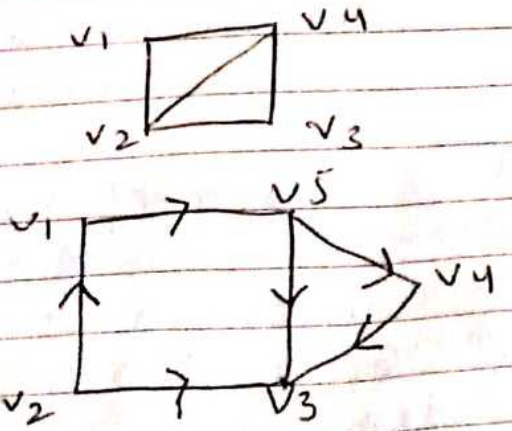
$$V(G) = (v_1, v_2, v_3 \ldots)$$

$$E(G) = (e_1, e_2, e_3 \ldots)$$

Types of graph

(1) Undirected

(2) Directed



Things To Do

Scanned by CamScanner

Terminology Used

(1) Weighted Graph A graph is said to be weighted graph if all the edges in it are labelled with some no.

(2) Self loop If there is an edge whose starting and end vertices are same.

(3) Parallel edges If there are more than one edges b/w the same pair of vertices then they are known as parallel edges.

(4) Adjacent vertices A vertex u is adjacent to other vertex v if there is an edge from u to v.

(5) Incidence In a graph the edge (u,v) is incident on vertices u and v.

(6) Degree of vertex The degree of a vertex is the no. of edges incident to that vertex.
In a directed graph there are 2 degrees for every node
(1) In degree → no. of edges coming
(2) Out degree → no. of edges going

25 SUNDAY
THINGS TO DO

(7) Simple graph A graph which does not have self loop or parallel edge.

(8) Multi graph A graph which has either self loop or parallel edges or both.

(9) Complete graph If each vertex is adjacent to every other vertex in graph.

(10) Regular Graph A graph is regular if every node is adjacent to the same no. of nodes.

(11) Planer graph A graph is planer if it can be drawn in a plane without intersecting any 2 edges.

(12) Connected graph In a graph G, 2 vertices v1 and v2 are said to be connected if there is a path in G from v1 to v2 or v2 to v1.

(a) strongly connected If for every pair of distinct vertices in G, there is a path.

(b) weakly connected If there is a path from v to u vertex but not u to v.

THINGS TO DO

Scanned by CamScanner

(13) Cycle. If there is a path containing one or more edges which start from a vertex and terminate onto the same vertex then the path is called a cycle.

(14) Acyclic Graph A graph does not have any cycle.

(15) Cyclic graph A graph which has cycle.

(16) Maximum edges in graph

Undirected → $n(n-1)/2$

Directed → $n(n-1)$

(17) Articulation Point If a node removed from graph, then graph becomes disconnected, then that node is called the articulation point.

(18) Bridge If an edge removed from graph, the graph becomes disconnected then that edge is called the bridge.

(19) Biconnected graph A graph with no articulation point

Representation of Graph

(1) set representation

(2) sequential Representation
   (a) Adjacenty matrix
   (b) Incidence matrix

(3) Linked Representation

(1) Set Representation In this representation we maintained 2 sets
   (1) Set of vertices [V(G)]
   (2) Set of Edges [V(E)]

(2) (a) Adjacency matrix It is the matrix which keeps the info. of adjacent nodes.

	v_1	v_2	v_3	v_4
v_1				
v_2				
v_3				
v_4				

$a_{ij} = \begin{cases} 1 & \text{if there is an edge from } v_i \text{ to } v_j \\ 0 & \text{otherwise} \end{cases}$

Scanned by CamScanner

29 2010 THURSDAY • APRIL

(b) Incidence matrix

Let G be a graph with n vertices and e edges then the incidence matrix of order n x e whose n rows correspond to the n vertices and e columns correspond to the e edges.

$$a_{ij} = \begin{cases} 1 & \text{if the } j^{th} \text{ edge } e_j \text{ is incident on } i^{th} \text{ vertex } v_i \\ 0 & \text{otherwise} \end{cases}$$

	e_1	e_2	e_3	e_4	e_5	...
v_1						
v_2						
v_3						
v_4						
:						
:						
:						

(3) Linked Representation

for non weighted graph

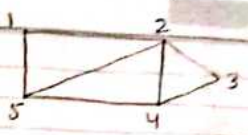
Info	Adj list

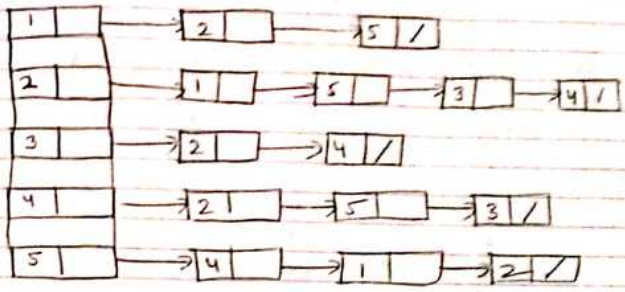
for weighted graph

Weight	Info	Adj list

THINGS TO DO

30 2010 APRIL • FRIDAY





Traversing of a graph

(1) BFS (BREADTH FIRST SEARCH)

(2) DFS (DEPTH FIRST SEARCH)

BFS Algorithm

1. Insert starting node into Queue.
2. Delete front element from the Queue and insert all its unvisited neighbours into the Queue at the end; and traverse them. Also make the value of visited array true for these nodes.
3. Repeat step 2 until Queue is empty.

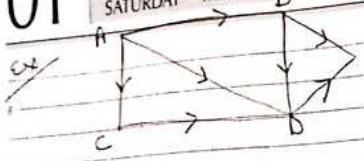
THINGS TO DO

Scanned by CamScanner

01

2010

SATURDAY • MAY



The Adjacency list of Graph is follo:

Vertex	Adjacency list
A	B, C, D
B	D, E
C	D
D	E
E	-

Step1 Push A into Queue.

front 0 1 2 3 4

F=0 | A | | | | - - -

R=0 Rear

Step2 Remove the A and Push all the Adjacent vertices of A

0 1 2 3 4

F=1 | | B | C | D | | - - -

R=3 F R

Traversed node A

2 SUNDAY

Step3 Pop the front element B and add the adjacent vertex of B which is not in Queue.

THINGS TO DO

0 1 2 3 4

F=2 | | | C | D | E | | - -

R=4 f R

Traversed Node A and B.

03

2010

MAY • MONDAY

Step4 Pop the front element from Queue and add the adjacent vertices of C which is not in Queue.

0 1 2 3 4

F=3 | | | | D | E | - -

R=4 F R

Traversed Node A, B, C

Step5 Pop the front element D and add the adjacent vertices of D, which are not in Queue.

0 1 2 3 4

F=4 | | | | | E | - -

R=4 F,R

Traversed Node A, B, C, D

Step6 Pop E and add vertices.

0 1 2 3 4 5

F=5 | | | | | | - -

R=4 R F

Traversed Node A, B, C, D, E It is the BFS of graph.

DFS Algorithm

(1) Push starting node into stack

THINGS TO DO

(2) Pop an element from stack. If it has not been traversed then traversed it otherwise Ignore it. After traversing make the value of visited

Scanned by CamScanner

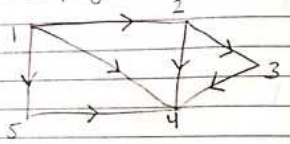


array true for this node.

3. Now push all the unvisited adjacent nodes of the popped element on stack. Push element even if it is already on the stack.

4 Repeat step 3 and 4 until stack is empty.

Ex



suppose the starting node is 1.

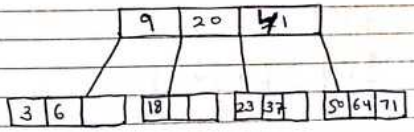
THINGS TO DO



B-Tree & its variants

m-way search tree are generalized version of binary search tree. The goals of M-way search tree is to minimize the accesses while retrieving a key from a file. B Trees are the balanced M-way search tree.

Trees having (M-1) keys & M children are called M-way search tree. A binary tree is a 2-way tree it means it has M-1 = 2-1 = 1 key & M=2 children in every node. A binary tree is also called an M-way tree of order 2. similarly an M-way tree of order 3 in which key values could be either 1 or 2 & it can have maximum 3 children.



→ In an M-way search tree all the nodes have degree ≤ M.

→ The keys in each node are in ascending order.

THINGS TO DO

→ The key Ki is larger than keys in subtree pointed by Pi and smaller than keys in subtree pointed by Pi+1.

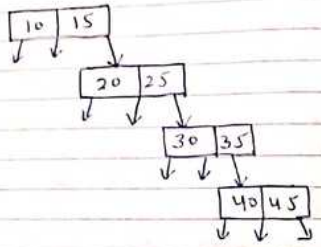
Scanned by CamScanner

06 2010 THURSDAY • MAY

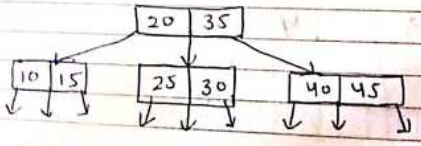
→ The subtrees are the M-way trees.

Insertion — 3 way search tree

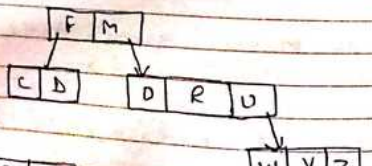
Ex (1) 10, 15, 20, 25, 30, 35, 40, 45



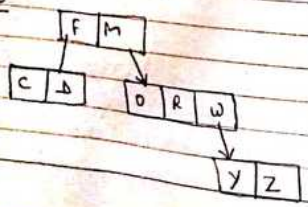
Ex 2 20, 35, 40, 10, 15, 25, 30, 45



Deletion



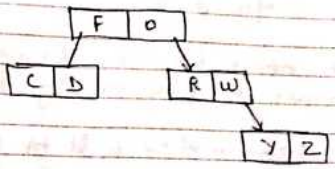
Delete U



THINGS TO DO

07 2010 MAY • FRIDAY

Delete M



Deletion

→ If (Ai = Aj = NULL) then Delete K.

→ If (Ai ≠ NULL and Aj = NULL) then choose largest of key element k' in child node pointed to by Ai delete the key k' and replace k by k'.

→ If (Ai = NULL and Aj ≠ NULL) then choose smallest of key element k'' from the subtree pointed to by Aj delete k'' and replace k by k''.

→ If (Ai ≠ NULL, Aj ≠ NULL) then choose either the largest or the smallest key element & replace.

THINGS TO DO

Scanned by CamScanner